VIETNAM NATIONAL UNIVERSITY, HANOI

INTERNATIONAL SCHOOL

REPORT OF FINAL EXAM PROJECT

Topic: Premier League football match prediction using Hidden Markov Model

Subject: Modeling

Class: INS3119

Lecturer: TS. Pham Dinh Tan

Group: 6

Student's name	Student's ID
Nguyen Minh Tue	22070923
Pham Thu Hang	21070676
Nguyen Manh Tuan	22071205
Tran Duy Thai	22071074

Hanoi, May 2026

TABLE OF CONTENT

LIST OF FIGURES

ABSTRACT

This project develops a probabilistic simulation system for the Premier League 2024–2025 season using Hidden Markov Models (HMM), Poisson Regression, and Monte Carlo Simulation. The study analyzes historical football data to model team performance dynamics, estimate match outcomes, and generate season-level predictions such as league standings, Top 4 probabilities, and relegation risks. The project combines concepts from probability theory, statistical modeling, and sports analytics to simulate realistic football season scenarios.

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our lecturer for providing valuable guidance, support, and knowledge throughout the completion of this project. Their advice and encouragement greatly contributed to the success of this study. We also would like to thank our university and faculty for providing the learning environment and resources necessary for conducting this research.

In addition, we are grateful to all team members for their cooperation, responsibility, and contributions during the development of the project.

DECLARATION

We hereby declare that this project entitled *"Premier League Football Match Prediction Using Hidden Markov Model"* is our own work and has been completed based on our research and understanding under the guidance of our lecturer.

All sources of information, references, and materials used in this project have been properly acknowledged and cited. This project has not been submitted for any other academic qualification or assessment at any institution.

We take full responsibility for the content and originality of this report.

CHAPTER 1: Project Information

1. Title of the project: Premier League football match prediction using Hidden Markov Model
2. Team members and roles

Nguyen Manh Tuan	- Data collection from multiple platforms - Main implementation of the simulation system (model development, data preprocessing, core coding) - Reviewing and improving technical content of the report	25%
Tran Duy Thai	- Supporting coding implementation - Assisting model development -Data processing and result presentation - Writing report sections - Preparing presentation slides	25%
Nguyen Minh Tue	- Supporting data collection - Dataset organization for analysis - Writing the project report - Preparing presentation slides.	25%
Pham Thu Hang	- Writing and refining the project report - Designing presentation slides - Supporting presentation structure -Contributing ideas and feedback during project development	25%

CHAPTER 2: Introduction

1. Why we choose this topic

Football season simulation is an interesting application of probability theory, mathematical modeling, and sports analytics. The Premier League is one of the most competitive football leagues in the world, with highly unpredictable match outcomes and large amounts of statistical data. These characteristics make it a valuable dataset for probabilistic analysis and football simulation studies.

Furthermore, this topic provides an opportunity to apply concepts from the Modeling course to real-world football data and sequential probabilistic systems.

3. Motivations

The main motivation of this project is to explore how probabilistic models can be used to simulate football league seasons under uncertainty. Football team performance changes dynamically throughout a season due to factors such as form, squad quality, and match conditions.

To model these uncertainties, the project applies Hidden Markov Models (HMM) to represent hidden team performance states, Poisson Regression to estimate goal distributions, and Monte Carlo Simulation to generate thousands of possible season outcomes.

The project also aims to strengthen practical skills in statistical modeling, data preprocessing, simulation techniques, and sports analytics using real football datasets.

3. Objectives and Scope

3.1. Objectives

The main objective of this project is to simulate the Premier League 2024–2025 season using probabilistic and statistical modeling techniques. The study aims to analyze team performance dynamics, estimate match-level outcomes, and generate season-level probability predictions.

Specific objectives include:

- Applying Hidden Markov Models (HMM) to model hidden team performance states.

- Using Poisson Regression to estimate expected goals and match outcomes.
- Applying Monte Carlo Simulation to generate multiple season scenarios.
- Evaluating league standings, Top 4 qualification probabilities, and relegation risks.
- Improving practical skills in football data analysis and probabilistic modeling.

3.2. Scope

This project focuses on offline simulation of the Premier League 2024–2025 season using historical football datasets and probabilistic methods.

The simulation considers important variables such as Expected Goals (xG), Elo ratings, squad quality (CA), team form, and rest time differences to model match outcomes and season performance.

The scope of the project is limited to season simulation and probabilistic analysis. Real-time prediction systems, live tracking systems, betting applications, and advanced deep learning models are not included in this study.

CHAPTER 3: Literature Review

1. Overview

This project is based on probabilistic and stochastic modeling methods for analyzing football match data over time. The main objective is to simulate season outcomes by modeling uncertainty in match results and team performance dynamics.

Football data is sequential in nature, where team performance changes across matches. Therefore, probabilistic models are suitable for capturing these time-dependent behaviors.

2. Markov Model

A Markov Model is a stochastic process in which the future state depends only on the current state, following the Markov property.

In football, team performance can be represented as a sequence of states that evolve over time based on recent match outcomes.

3. Hidden Markov Model (HMM)

The Hidden Markov Model is an extension of the Markov process in which the system states are not directly observable.

In this framework:

- Hidden states represent latent team performance conditions
- Observations represent match statistics and results

An HMM consists of:

- Initial state probabilities
- State transition probabilities
- Emission probabilities

The model learns how teams transition between performance states over time and how these states generate observable match outcomes.

4. Poisson Regression

Poisson regression is a statistical method used for modeling count-based data. It is commonly applied to estimate the expected number of goals in football matches.

The model assumes that goal counts follow a Poisson distribution and uses input variables such as team strength and performance indicators to estimate expected goals.

5. Monte Carlo Simulation

Monte Carlo Simulation is a numerical method used to model uncertainty by generating a large number of random outcomes.

In football modeling, it is used to simulate multiple possible match results based on predicted goal distributions, allowing estimation of probabilities for outcomes such as win, draw, or loss, as well as season-level results.

6. Data Processing and Evaluation

Data preprocessing is an essential step in preparing sequential football data for modeling. Common techniques include feature scaling, rolling statistics, and handling missing values.

Model performance is evaluated through simulation consistency and probabilistic output analysis rather than deterministic accuracy, due to the stochastic nature of football outcomes.

CHAPTER 4: Project Implementation and Results

1. Data collection & preprocessing

1.1 Data Sources

To ensure the simulation engine operates with high accuracy, data collection was conducted across multiple specialized platforms(ClubElo, FBref, API-Football, FM24, Kaggle).The foundational variable rating of matches of all 20 Premier League teams was established using real-time Elo rating fetched via the API of ClubElo.

Advanced team metrics focused primarily on Expected Goals (xG), with datasets sourced from Kaggle that originally derived from FBref. These datasets provide detailed match-level performance metrics such as performance, playing time,… most noticeable are xG values, which are important for estimating attacking and defensive quality. In this project, xG was separated into home and away values (home_xG and away_xG) to better capture the Home Advantage effect and provide expected goal parameters (λ) for the Bivariate Poisson Regression model.

However, the Kaggle datasets mainly contain historical match results(indicators) and do not fully capture scheduling-related factors such as fixture congestion, or participation in multiple competitions. To address this limitation, API-Football was used to collect complete fixture schedules across the Premier League, FA Cup, and UEFA competitions. This additional scheduling data allowed the project to calculate “Rest Days” and model physical fatigue throughout the season. By using API-Football data, the simulation engine can better represent the impact of short recovery periods and congested match schedules on team performance.

For individual player metrics, a database of Football Manager 2024 was utilized to extract "Current Ability" (CA) scores, which mathematically model the individual talent levels within the team. Although it is measured at the individual player level, the project gives you squad quality by calculating the average CA of each team's Top 15 highest-rated players. And thus, we are able to get the variable that represents the team's overall ability. In reality, a football match typically involves around 11 starting players and a few substitute players(most commonly 4). This approach allows the model to better represent the practical competitive strength of a team while reducing distortion from youth or reserve players with limited match impact.

```
# Calculate the Top 15 CA Average per team
squad_ca_dict = {}
for team, group in df_ca_clean.groupby('Standard_Club'):
    top_15 = group.sort_values(by='CA', ascending=False).head(15)
    squad_ca_dict[team] = round(top_15['CA'].mean(), 2)
```

Figure 1: Calculate the Top 15 CA Average per team

1.2 Data Cleaning

The raw datasets required strict standardization before integration to ensure accurate indexing. This involved resolving URL string variations from the ClubElo API (e.g., mapping 'ManUnited' to 'Manchester Utd') and correcting naming inconsistencies that were from Football Manager 2024 database(e.g., standardizing 'Manchester UFC: Manchester Utd') .

To prevent simulation crashes, missing data in the 'Rest Days' variable was handled by assigning a default value. For example, unrecorded intervals of the season's opening matches were processed by applying a 14-day fully-rested state using .fillna(14). Additionally, prolonged gaps like international breaks were standardized using the .clip(upper=14) function. This caps the maximum rest value at 14 days, treating any extended recovery period as a complete physical

reset.. # Any rest longer than 14 days (e.g., international breaks) acts as a full reset.

```
# For the first game of the season, rest days will be NaN. Cap it at 14 days (fully rested baseline).
df_sched['Rest_Days'] = df_sched['Rest_Days'].fillna(14)
# Any rest longer than 14 days (e.g., international breaks) acts as a full reset.
df_sched['Rest_Days'] = df_sched['Rest_Days'].clip(upper=14)
```

Figure 2: Rest Days Normalization and Fatigue Handling in Match Scheduling Data

Non-competitive friendly matches were also dropped from the API-Football dataset to ensure the resulting 'Rest Days' fatigue penalty only captured the physical toll of competitive fixtures.

```
# Drop friendlies (they don't count towards competitive physical fatigue)
df_sched = df_sched[df_sched['Competition'] != 'Friendlies Clubs'].copy()
```

Figure 3: Removal of Non-Competitive Matches from Dataset

Once cleaned, the raw data was transformed into the specific inputs required by the simulation engine. The “Rest Days” (understood as Fatigue Penalty) variable were aligned to reflect each team's exact state at the start of the 2024/2025 season. Using multi-competition schedules sourced from API-Football, the model quantified fixture congestion and calculated its physical impact on team performance. Combined with the Fatigue Penalty variable, this allowed the simulation engine to account for the effects of insufficient recovery time on match outcomes.

1.3 Dataset Merging and Architecture

The final phase of the data pipeline involved integrating the four distinct datasets into a unified, simulation-ready architecture. The integration utilized a standardized 'Gold Standard' team naming convention as the primary join key to ensure data integrity across the ClubElo, API-Football, FM24, and FBref sources. Specifically clubelo_fetch.ipynb for ratings, eu_n_domestic.ipynb for

multi-competition schedules, and clean_data.ipynb for the main pipeline—all variables were merged into the final master_simulation_matrix.csv.

The architecture was designed to transition from raw data team-level to match-level differential features. During the merging process, the engine calculated three critical delta variables for each fixture in the 2024/2025 schedule:

- **Elo_Diff:** The difference in team strength based, team's form on Club Elo ratings between the home and away teams.
- **CA_Diff:** The difference in squad quality calculated from the average Current Ability (CA) ratings of each team's top 15 players.
- **Rest_Diff:** The difference in recovery time between teams, measured using the number of rest days between fixtures across all competitions.

This master matrix ensures that every fixture row contains the exact Elo, CA, schedule density, and xG parameters required as inputs for the Bivariate Poisson Regression model, enabling efficient batch processing during Monte Carlo simulations.

2. Variables and Assumptions

2.1 Key Variables

The simulation engine relies on several core variables to estimate match outcomes and season performance. These variables were selected to capture differences in team quality, tactical strength, player(team) current ability, and physical condition throughout the 2024/2025 season.

- **Elo_Diff:** Represents not only the strength of a team but also its recent form and momentum. If a team's current Elo rating is significantly higher than its initial season rating, this generally indicates strong recent performances and positive form.

- **CA_Diff:** Measures team quality differences using the average Current Ability (CA) ratings of each team's top 15 players extracted from the Football Manager 2024 database. This variable represents the overall talent level and technical quality of a team.
- **Rest_Diff:** Measures the fatigue differential between teams based on differences in recovery days between competitive fixtures. Teams with shorter recovery periods are assumed to experience higher physical fatigue and reduced performance efficiency.
- **home_xG and away_xG:** Represent the Expected Goals (xG) values generated by teams in home and away matches. These metrics estimate the probability of scoring opportunities created during matches and are used as offensive performance indicators within the simulation model.

*Team "form" in the project is not determined solely by Elo ratings. The simulation engine also evaluates performance efficiency using Expected Goals (xG) and Expected Goals Against (xGA). A team with high xG values but a low number of actual goals scored indicates poor finishing efficiency and weak attacking form, as the team fails to convert scoring opportunities into goals. Similarly, if a team records low xGA values but still receives many goals, this suggests poor defensive efficiency or underperformance relative to the quality of chances conceded. By combining Elo dynamics with xG and xGA efficiency analysis, the model can represent team form more realistically instead of relying only on match results or rating changes.

Together, these variables form the primary inputs for the Bivariate Poisson Regression model and directly influence the expected goal values (λ) used during match simulations.

2.2 Modeling Assumptions

The simulation engine operates under several football-related assumptions to simplify the complexity of real-world match outcomes. Teams with higher Elo

ratings and stronger team quality generally have a greater probability of winning matches. Similarly, higher Expected Goals (xG) values are interpreted as indicators of stronger attacking performance and better chance creation efficiency during matches.

The project also assumes that physical fatigue has a measurable impact on team performance throughout the season. Teams participating in busy schedules with shorter recovery periods are expected to experience reduced physical efficiency, which may negatively affect match results. In contrast, teams with longer recovery periods are assumed to perform closer to their optimal level.

Finally, the simulation makes the assumption that home teams possess an advantage due to factors such as crowd support, stadium familiarity(for such as home/away variables). Team momentum and seasonal form are also assumed to fluctuate dynamically over time, allowing the simulation engine to better represent the uncertainty and variability commonly observed in real football competitions.

3. Data Preprocessing

3.1 Rolling Statistics

One of the main preprocessing steps in the project is the calculation of rolling statistics. The system uses the previous five matches of each team to estimate recent performance and momentum before a new match is simulated or predicted.

```
# We calculate the rolling sum of points and mean of xGD over the PREVIOUS 5 matches.
# We use .shift(1) because we want their form BEFORE the match starts.
team_log['Rolling_Pts'] = team_log.groupby('Team')['Points'].transform(lambda x: x.rolling(5, min_periods=1).sum().shift(1))
team_log['Rolling_xGD'] = team_log.groupby('Team')['xGD'].transform(lambda x: x.rolling(5, min_periods=1).mean().shift(1))

# For the opening games of the season, we pad them with a strict "Baseline" expectation
# (e.g., averaging 1 point per game, 0.0 xG difference)
team_log['Rolling_Pts'] = team_log['Rolling_Pts'].fillna(5.0)
team_log['Rolling_xGD'] = team_log['Rolling_xGD'].fillna(0.0)


 # We calculate the rolling sum of points and mean of xGD over the PREVIOUS 5 matches.
 # We use .shift(1) because we want their form BEFORE the match starts.
 team_log['Rolling_Pts'] = team_log.groupby('Team')['Points'].transform(lambda x: x.rolling(5, min_periods=1).sum().shift(1))
 team_log['Rolling_xGD'] = team_log.groupby('Team')['xGD'].transform(lambda x: x.rolling(5, min_periods=1).mean().shift(1))
```

Figure 4: Rolling statistics

The rolling window calculates the total points collected over the last five matches. This allows the model to measure short-term form instead of relying only on season-long statistics.

The .shift(1) operation is very important because it prevents data leakage. Without this operation, the current match result could accidentally be included in the prediction features, causing unrealistic model performance. By shifting the rolling values by one match, the system only uses information available before the current game.

This preprocessing method helps the prediction models better represent team momentum and recent performance trends.

3.2 Expected Goal Difference

The project also calculates Expected Goal Difference (xGD), which is used as a performance feature for prediction models.

```
# Calculate Points and Match xGD
team_log['Points'] = np.where(team_log['GF'] > team_log['GA'], 3, np.where(team_log['GF'] == team_log['GA'], 1, 0))
team_log['xGD'] = team_log['xGF'] - team_log['xGA']
```

Figure 5: xGD calculation

Expected goals (xG) estimate the quality of scoring chances created by a team. By subtracting expected goals conceded from expected goals scored, the system generates an overall performance indicator for each team.

Positive xGD values usually indicate stronger attacking and defensive performances, while negative values may indicate weaker overall performance.

3.3 Missing Value Handling

Rolling calculations and sequential preprocessing operations can generate missing values, especially during the first matches of a season where insufficient historical data exists.

To handle this issue, the project replaces missing values using default numerical values.

```
# For the opening games of the season, we pad them with a strict "Baseline" expectation
# (e.g., averaging 1 point per game, 0.0 xG difference)
team_log['Rolling_Pts'] = team_log['Rolling_Pts'].fillna(5.0)
team_log['Rolling_xGD'] = team_log['Rolling_xGD'].fillna(0.0)
```

Figure 6: Missing Value Handling

This preprocessing step stabilizes the model and prevents runtime errors during training and simulation.

In this project, the default value 5.0 is used to simulate an average recent form for teams that do not yet have enough historical match data, such as newly promoted clubs at the beginning of the season. The value roughly represents a neutral performance level similar to drawing several recent matches.

Handling missing values is important because machine learning models cannot process NaN values directly.

3.4 Simulation Configuration Variables

The project uses several configuration variables to control the behavior of the prediction and simulation models.

```
# 1. Simulation Scope
SIMULATION_ITERATIONS = 1000
ELO_K_FACTOR = 20  # Standard Elo volatility (higher = more volatile rankings)

# 2. HMM Form Thresholds (The xGD required to trigger a hot/cold streak)
FORM_HOT_THRESHOLD = 0.5
FORM_COLD_THRESHOLD = -0.5

# 3. Approach B: "Individual Brilliance" Bailout Parameters
# Tier 1 (Elite Squads)
BAILOUT_CA_ELITE = 155.0     # City, Arsenal, Liverpool, Utd
BAILOUT_CHANCE_ELITE = 0.50  # 50% chance

# Tier 2 (Strong Squads)
BAILOUT_CA_STRONG = 145.0    # Chelsea, Spurs, Newcastle, Villa, Brighton
BAILOUT_CHANCE_STRONG = 0.25 # 25% chance

# The Rest of the League (Base Resilience)
BAILOUT_CHANCE_REST = 0.10   # 10% chance for everyone else
```

Figure 7:Model Configuration Parameters

The variable SIMULATION_ITERATIONS defines the number of simulated seasons generated during Monte Carlo simulation. Running a larger number of simulations helps produce more stable probability estimates.

ELO_K_FACTOR controls the volatility of Elo rating updates. Higher values cause team ratings to change more aggressively after matches.

The project also defines Hidden Markov Model form thresholds using expected goal difference (xGD). Teams with xGD values above the hot threshold are considered in strong form, while teams below the cold threshold are considered in poor form.

In addition, bailout configuration variables are introduced to simulate the impact of high-quality squads. Elite teams are given higher recovery probabilities when entering poor form states, helping the simulation better represent real football behavior where strong squads can recover more consistently.

4. Feature Engineering

4.1 Elo Difference

The project uses Elo rating differences as one of the main predictive features.

```
'Elo_Diff': home_elo - away_elo,
```

Figure 8: Elo Difference

The Elo rating system is commonly used to estimate relative team strength. Higher Elo ratings generally represent stronger teams based on historical performance.

The calculated Elo difference measures the strength gap between the home and away teams before a match begins. Positive values indicate that the home team is stronger, while negative values suggest an advantage for the away team.

4.2 CA Difference

The project also uses Current Ability Difference (CA Difference) as a feature representing squad quality differences between teams.

```
'CA_Diff': home_ca - away_ca,
```

Figure 9: CA Difference

Current Ability (CA) ratings are based on Football Manager squad evaluations. Higher CA values generally indicate stronger squads with better overall player quality and depth.

The model calculates the difference between the home team CA and away team CA before each match. Positive values indicate that the home team has a stronger squad, while negative values suggest an advantage for the away team.

Including CA Difference helps the prediction model better represent squad strength and player quality differences during match simulations.

4.3 Rest Difference

The project also uses Rest Difference as a feature to measure the recovery time difference between two teams before a match.

```
'Rest_Diff': home_rest - away_rest
```

Figure 10: Rest Difference

Teams with longer recovery periods may perform better due to reduced fatigue, improved physical condition, and better tactical preparation.

The model calculates the difference between the home team rest days and away team rest days before each match. Positive values indicate that the home team has more recovery time, while negative values indicate an advantage for the away team.

Including Rest Difference helps the simulation better represent scheduling effects and fixture congestion throughout the football season.

4.4 Form Difference

The project also calculates form difference to compare recent team momentum.

```
df['Form_Diff'] = df['Home_Form'] - df['Away_Form']
```

Figure 11: Form difference

This feature compares the recent performance levels of both teams using rolling statistics generated during preprocessing.

Teams with strong recent form are more likely to achieve better match results. By including form difference as an input feature, the prediction models can better capture short-term momentum changes throughout the season.

5. Model Implementation

5.1 Hidden Markov Model

The project uses a Gaussian Hidden Markov Model to analyze hidden team momentum states.

```
# Initialize the HMM to find exactly 3 hidden states
model = hmm.GaussianHMM(n_components=3, covariance_type="diag", n_iter=1000, random_state=42)
model.fit(X)
```

Figure 12: Hidden Markov Model

The parameter n_components=3 represents three hidden performance states used by the model. The HMM processes sequential football data and attempts to identify hidden patterns in team performance over time.

The model analyzes rolling features such as recent points and expected goal difference (xGD) to learn hidden momentum patterns throughout the season. The hidden states represent different team conditions such as Hot form, Baseline form, and Cold form.

Unlike traditional machine learning models, Hidden Markov Models are designed for sequential data. This makes them suitable for football analytics because team form changes continuously throughout a season.

The generated hidden states are later used as additional features for match prediction and simulation.

```
HMM Output Sample for Chelsea:
           Date  Rolling_Pts  Rolling_xGD  Form_State
223  2025-04-26          8.0         0.32           2
224  2025-05-04         11.0         0.46           1
225  2025-05-11         11.0         0.86           2
226  2025-05-16         10.0         0.66           1
227  2025-05-25         12.0         0.54           2
```

Figure 13:Hidden Form States Generated by HMM

5.2 Poisson Regression

Poisson regression is used to predict the number of goals scored by each team.

```python
# Model 1: Predicting Home Goals
home_model = smf.glm(formula="Home_Goals ~ Elo_Diff + CA_Diff + Rest_Diff + Form_Diff",
                     data=df,
                     family=sm.families.Poisson()).fit()

# Model 2: Predicting Away Goals.
away_model = smf.glm(formula="Away_Goals ~ Elo_Diff + CA_Diff + Rest_Diff + Form_Diff",
                     data=df,
                     family=sm.families.Poisson()).fit()
```

Figure 14:Poisson Regression Model Implementation Code

Football goals are count-based events, making Poisson regression suitable for this type of prediction problem.

The model estimates expected goal values using several input features such as Elo difference, recent form, and team momentum.

Separate models are created for home goals and away goals. The predicted goal values are later used in Monte Carlo simulation to generate match outcomes.

```
--- Home Model Beta Weights (β) ---
Intercept    0.361507
Elo_Diff     0.000335
CA_Diff      0.012358
Rest_Diff    0.009590
Form_Diff    0.127714
dtype: float64
```

Figure 15: Poisson Regression Model Coefficients

5.3 Monte Carlo Simulation

The project uses Monte Carlo simulation to generate probabilistic football match outcomes.

The simulation repeatedly generates match results by sampling from Poisson distributions using predicted expected goal values from the Poisson regression models.

```
# Run 10,000 iterations
home_simulated_goals = np.random.poisson(home_lambda, simulations)
away_simulated_goals = np.random.poisson(away_lambda, simulations)
```

Figure 16:Monte Carlo Simulation

By simulating thousands of possible matches, the system can estimate:

- Home win probability
- Draw probability
- Away win probability
- Most likely scorelines

Monte Carlo simulation helps model the uncertainty of football matches because real football games are highly unpredictable.

5.4 Dynamic Team Form Update

The project dynamically updates team form during simulation using the update_team_form() function.

```
def update_team_form(team, current_state, sim_xgd, squad_ca):
    # 1. Base HMM Logic using Control Panel variables
    if sim_xgd > FORM_HOT_THRESHOLD:
        next_state = 2 # Hot
    elif sim_xgd < FORM_COLD_THRESHOLD:
        next_state = 0 # Cold
    else:
        next_state = 1 # Baseline

    # 2. The Bailout Logic
    if next_state == 0:
        team_ca = squad_ca.get(team, 130)

        # Check Elite (50%)
        if team_ca >= BAILOUT_CA_ELITE:
            if np.random.rand() < BAILOUT_CHANCE_ELITE:
                return 1

        # Check Strong (25%)
        elif team_ca >= BAILOUT_CA_STRONG:
            if np.random.rand() < BAILOUT_CHANCE_STRONG:
                return 1

        # The Rest of the League (10%)
        else:
            if np.random.rand() < BAILOUT_CHANCE_REST:
                return 1

    return next_state
```

Figure 17:Dynamic Team Form Update Function

The function evaluates the simulated expected goal difference (sim_xGD) to determine the next form state of each team. Teams are classified into three states: Cold (0), Baseline (1), and Hot (2).

If the simulated xGD exceeds the hot threshold, the team transitions to a Hot state. If the xGD falls below the cold threshold, the team enters a Cold state. Otherwise, the team remains in the Baseline state.

The function also implements a “bailout logic” mechanism. Teams with higher squad quality (measured by Current Ability - CA) have a probability of recovering from poor form states. Elite squads have a 50% recovery chance, strong squads have a 25% chance, while other teams have a 10% chance.

This mechanism improves realism by reflecting the ability of stronger teams to recover from temporary poor performance due to better squad depth and quality.

The dynamic update system allows team form to evolve throughout the simulation, making the season model more realistic and adaptive.

6. Simulation Results

The project evaluates overall season outcomes using Monte Carlo simulation.

```
======================================
               Team  Avg_Points  Avg_GD
1   Manchester City        82.2    48.9
2           Arsenal        77.4    40.4
3         Liverpool        73.0    33.2
4    Manchester Utd        61.5    13.8
5           Chelsea        59.1    10.6
6     Newcastle Utd        56.8     6.8
7         Tottenham        56.8     6.6
8       Aston Villa        55.4     3.9
9           Brighton        53.0     0.7
10          West Ham        49.3    -5.4
11   Crystal Palace        48.9    -6.1
12        Brentford        47.4    -8.1
13           Fulham        46.3   -10.4
14      Bournemouth        44.6   -13.3
15          Everton        43.8   -14.3
16     Nottm Forest        43.5   -14.1
17           Wolves        40.9   -18.8
18   Leicester City        39.3   -21.5
19      Southampton        37.7   -24.3
20     Ipswich Town        35.6   -28.6
======================================
Simulation completed in 12.19 seconds.
```

Figure 18: League table

The final simulated league table shows the average points and goal difference of each team after running multiple season simulations.

```
================================================================================
 ADVANCED MONTE CARLO ANALYTICS (1,000 UNIVERSES)
================================================================================
              Team  Avg Pts  Max Pts  Min Pts Title % Top 4 % Relegation %
1   Manchester City     82.2      108       54   50.9%   95.4%         0.0%
2           Arsenal     77.4      110       44   26.4%   88.2%         0.0%
3         Liverpool     73.0      100       35   15.0%   76.3%         0.1%
4    Manchester Utd     61.5       91       34    2.3%   32.0%         0.5%
5           Chelsea     59.1       96       25    1.7%   23.3%         1.9%
6         Tottenham     56.8       93       23    0.7%   17.3%         2.1%
7     Newcastle Utd     56.8       92       26    1.3%   18.6%         3.5%
8       Aston Villa     55.4       88       23    0.8%   14.3%         3.5%
9          Brighton     53.0       83       20    0.7%   10.1%         5.8%
10         West Ham     49.3       80       20    0.1%    5.4%        10.4%
11   Crystal Palace     48.9       80       19    0.1%    6.1%        11.1%
12        Brentford     47.4       83       16    0.0%    3.3%        11.9%
13           Fulham     46.3       90       19    0.0%    2.4%        16.1%
14      Bournemouth     44.6       80       19    0.0%    1.7%        20.3%
15          Everton     43.8       74       10    0.0%    1.7%        22.4%
16     Nottm Forest     43.4       75       18    0.0%    1.6%        23.5%
17           Wolves     40.9       73        9    0.0%    0.7%        30.6%
18   Leicester City     39.3       75       13    0.0%    0.7%        39.7%
19      Southampton     37.7       75       12    0.0%    0.6%        42.7%
20     Ipswich Town     35.6       69       11    0.0%    0.3%        53.9%
================================================================================
```

Figure 19:Season analytics

The project generates season-level statistics after running multiple Monte Carlo simulations of the league.

The output table includes key performance indicators such as average points, maximum and minimum points, and probabilistic outcomes including title, top-four, and relegation probabilities.

The “Avg Pts” column represents the expected points accumulated by each team across all simulated seasons. “Title %” indicates the probability of winning the league, “Top 4 %” represents qualification probability for Champions League positions, and “Relegation %” estimates the risk of finishing in the relegation zone.

These results are derived from thousands of simulated league outcomes, allowing the model to capture uncertainty and variability in long-term team performance.

CHAPTER 5: Conclusion

1. Conclusion

This project developed a probabilistic simulation system for the Premier League 2024–2025 season using Hidden Markov Models, Poisson Regression, and Monte Carlo Simulation. The system models team performance as a dynamic process by extracting latent form states from rolling match statistics and using them to support match-level prediction.

Expected goals are estimated using a Poisson-based model with key differential features including Elo rating, squad quality, rest time, and form state. Monte Carlo Simulation is then used to generate multiple season scenarios and produce probabilistic league outcomes such as final standings, title chances, Top 4 qualification, and relegation risk.

Overall, the system demonstrates how probabilistic and sequential modelling techniques can be applied to simulate complex football competitions under uncertainty.

2. Future Work

Several improvements can be made to enhance the system in future research.

First, the current Poisson model assumes independence between home and away goals. This can be improved by using a Bivariate Poisson model with correlation adjustment to better capture match-level dependencies.

Second, the current form update mechanism in simulation is based on rule-based thresholds. This can be replaced with learned transition probabilities from the Hidden Markov Model to make form evolution more data-driven.

Third, additional features such as injuries, managerial changes, weather conditions, and player-level statistics could be integrated to improve model realism.

Finally, the system can be extended to support real-time or rolling-season prediction, allowing continuous updates after each matchweek instead of a single pre-season simulation.

CHAPTER 6: Reference

datasets: https://www.kaggle.com/datasets/furkanark/premier-league-2024-2025-data?resource=download

Tools:

FMRTE 24: https://www.fmrte.com/files/file/63-fmrte-24-for-windows/ (for CA extraction)

project reference: https://github.com/jbaranski/mls-hmm

ClubElo. (2026). *Football club Elo ratings*. Retrieved May 14, 2026, from https://clubelo.com/

Opta Analyst. (2023). *What is Expected Goals (xG)?* Retrieved May 14, 2026, from https://theanalyst.com/eu/2023/08/what-is-expected-goals-xg/

Sports Interactive. (2026). *Football Manager 2024*. Retrieved May 14, 2026, from https://www.footballmanager.com/games/football-manager-2024

Hudl Statsbomb. (2024). *Expected Goals and Expected Goals Against explained*. Retrieved May 14, 2026, from https://www.hudl.com/blog/expected-goals-xg-explained

Jason M. Kinser. (2022). *Modeling and Simulation in Python*. Shared by lecturer via Microsoft Teams.